Aggregation and Grouping

Python Data Science Handbook — Chapter 3, Section 3.8

From whole-table summaries to per-group analytics: the split–apply–combine engine at the heart of Pandas.

1 Why this section matters

Background & Context

A huge fraction of "data analysis" is really just two questions repeated over and over: *what is the overall summary?* and *how does that summary differ across categories?* The first question is answered by *aggregation* — collapsing many numbers into one (a sum, a mean, a maximum). The second is answered by *grouping* — splitting the data into buckets by some key and aggregating within each bucket.

If you have ever written a SQL `GROUP BY`, or dragged a field into the "rows" shelf of a pivot table, you already know the shape of this idea. Pandas gives you the same power in code, but with two advantages: the result is itself a labeled Pandas object you can keep computing on, and the "apply" step can be an *arbitrary* Python function, not just a fixed menu of SQL aggregates.

The convention we use throughout:

```
import numpy as np
import pandas as pd
```

2 Simple aggregations: collapsing an axis

Before grouping, get comfortable with plain aggregation. Every `Series` and `DataFrame` carries a family of reduction methods — **`sum`**, `mean`, **`min`**, **`max`**, `std`, `var`, `median`, `prod`, `count` — each of which takes many values and returns few.

```
rng = np.random.default_rng(42)
s = pd.Series(rng.random(5))
print(s.sum(), s.mean())
```

```
2.5613... 0.5122...
```

For a `DataFrame` the interesting wrinkle is the `axis` argument, because now there are *two* directions you could collapse.

```
df = pd.DataFrame({'A': rng.random(4),
                   'B': rng.random(4)})
df.mean()          # default axis=0: collapse the ROWS, one value per
    column
```

```
df.mean(axis=1)    # axis=1: collapse the COLUMNS, one value per row
```

Intuition

The `axis` argument names the axis that gets *eaten*, not the one that survives. `axis=0` is the row axis, so `df.mean(axis=0)` consumes the rows and leaves one number per *column*. This is the opposite of what many people guess on their first try. A useful phrasing: "`axis=0` means *down the rows*; `axis=1` means *across the columns*."

Common Pitfall

Aggregations **skip `NaN` by default** (`skipna=True`). That is usually what you want, but it means `df.mean()` and `df.sum()` silently ignore missing values rather than propagating them. If a column is half missing, its reported mean is the mean of the half that exists — not flagged, not warned. When that matters, check `df.count()` (which counts *non-null* entries per column) alongside your aggregate.

2.1 `describe()`: the one-line reconnaissance

When you first meet a dataset, `describe()` runs a whole panel of aggregations at once for every numeric column:

```
df.describe()
```

```
              A         B
count  4.000000  4.000000
mean   0.546...  0.471...
std    0.233...  0.301...
min    0.218...  0.122...
25%    0.451...  0.275...
50%    0.602...  0.470...
75%    0.697...  0.666...
max    0.760...  0.823...
```

That single call gives you count, mean, spread, and the five-number summary — enough to spot an obviously broken column (a `max` of 10^9, a `count` far below the others) in seconds.

> **Cheat sheet — the common reductions:**
> `count` (non-null), `sum`, `mean`, `median`, `min`, `max`,
> `std`, `var`, `prod`, `first`, `last`, `nunique`.

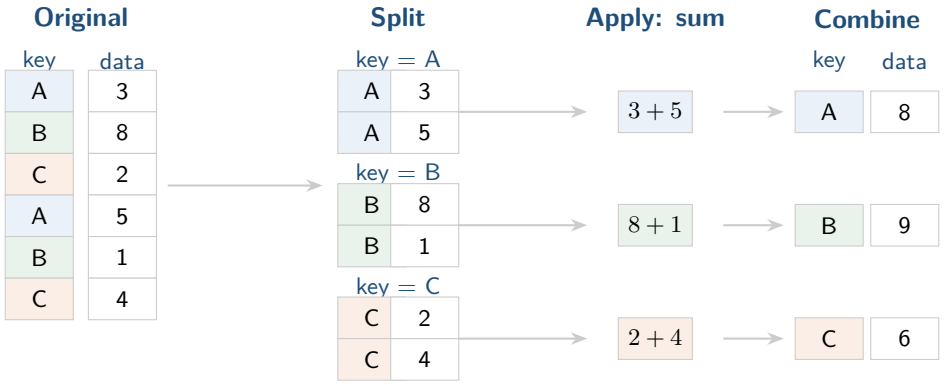
3 The big idea: split–apply–combine

Whole-table aggregation throws away all the structure in your data. The real power comes when you aggregate *conditionally* — separately for each value of some key. The pattern that organizes this is called **split–apply–combine**, a name coined by Hadley Wickham (of R's `plyr`/`dplyr` fame). It breaks every grouped computation into three crisp phases.

Key Idea

Split the rows into groups according to the values of a key. **Apply** a function (usually an aggregation) independently within each group. **Combine** the per-group results back into a single output object, indexed by the group keys. Holding these three phases separate in your mind is the single most useful thing you can do when reasoning about `groupby`.

Consider a tiny table whose key takes the values `A`, `B`, `C`, and whose `data` column holds some numbers. We want the *sum within each key*. The diagram below traces all three phases.



In code, this entire diagram is one line:

```
df = pd.DataFrame({'key':  ['A', 'B', 'C', 'A', 'B', 'C'],
                   'data': [3, 8, 2, 5, 1, 4]})
df.groupby('key').sum()
```

```
     data
key
A       8
B       9
C       6
```

Notice that the group key has become the *index* of the result. That is the "combine" phase choosing a sensible label for each row.

4 groupby is lazy

Here is a detail that surprises people. The call `df.groupby('key')` by itself does *not* compute anything:

```
df.groupby('key')
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x...>
```

It hands back a `DataFrameGroupBy` object: a lightweight bundle that has merely *worked out which rows belong to which group*, but has applied no function and collapsed nothing. The actual

computation is deferred until you call an aggregation on it.

Intuition

Think of the `GroupBy` object as a *recipe*, not a *meal*. It knows how to slice the data into groups, and it is waiting to hear what you want done to each slice. Because nothing is computed yet, Pandas can be clever: it does not have to physically split the frame into a pile of sub-frames in memory. When you finally say `.sum()`, it can run a single fused pass over the data. This laziness is what makes `groupby('key').sum()` efficient even on large frames, and it is what lets you write fluent one-liners that read like the split–apply–combine story they implement.

5 Living with a GroupBy object

Even though it is lazy, the `GroupBy` object supports a rich interface. Three capabilities show up constantly.

5.1 Column indexing: aggregate just one column

A `GroupBy` can be indexed by column name, exactly like the `DataFrame` it came from. This narrows the apply step to the columns you care about and is the idiomatic way to aggregate a single series by a key:

```
df.groupby('key')['data'].mean()
```

```
key
A    4.0
B    4.5
C    3.0
Name: data, dtype: float64
```

This is itself still lazy: `df.groupby('key')['data']` returns a `SeriesGroupBy` object, and nothing is computed until `.mean()`.

5.2 Iteration: groups as (name, sub-frame) pairs

You can loop over a `GroupBy`. Each iteration yields a tuple of *(group key, the sub-DataFrame for that key)*:

```
for key, sub in df.groupby('key'):
    print(key, 'has', len(sub), 'rows')
```

```
A has 2 rows
B has 2 rows
C has 2 rows
```

Common Pitfall

Iterating in Python and processing groups one at a time *works*, but it throws away the very efficiency that the lazy, vectorized `GroupBy` was built to give you. Reach for explicit iteration only for genuinely custom, hard-to-vectorize logic. For anything expressible as an aggregation, filter, or transform (the next sections), let Pandas do it in one fused pass instead.

5.3 Method dispatch: per-group convenience methods

Any `DataFrame` method that the `GroupBy` does not define itself is *dispatched* to each group and the results combined. The classic example is `describe()`:

```
df.groupby('key')['data'].describe()
```

```
     count  mean       std  min   25%  50%   75%  max
key
A      2.0   4.0  1.414214  3.0  3.50  4.0  4.50  5.0
B      2.0   4.5  4.949747  4.5   ...  4.5   ...  8.0
C      2.0   3.0  1.414214  2.0  2.50  3.0  3.50  4.0
```

You never wrote a loop, yet you got a full statistical summary *per group*. That is dispatch plus combine working together.

6 The four operation families

Once you have a `GroupBy` object, there are four broad things you can do with it. Keeping their *output shapes* straight is the key to using them correctly.

Operation	What you give it	Shape of result
`aggregate`	reduction(s) per column	one row per group
`filter`	group → True/False	subset of original rows
`transform`	group → same-shape	same shape as input
`apply`	group → anything	whatever you return

We will set up a slightly richer frame to exercise all four:

```
df = pd.DataFrame({'key':  ['A', 'B', 'C', 'A', 'B', 'C'],
                   'val1': [0, 1, 2, 3, 4, 5],
                   'val2': [5, 0, 3, 3, 7, 9]})
```

6.1 aggregate: one or more reductions

`aggregate` (alias `agg`) generalizes the simple `.sum()` call. You can hand it a single function, a *list* of functions (to compute several reductions at once), or a *dict* mapping columns to the functions you want for each.

```
# A list of reductions, applied to every column
df.groupby('key').aggregate(['min', 'median', 'max'])
```

```
    val1            val2
     min median max   min median max
key
A      0    1.5   3     3    4.0   5
B      1    2.5   4     0    3.5   7
C      2    3.5   5     3    6.0   9
```

A list of functions produces a *hierarchical* (MultiIndex) set of columns — the outer level names the original column, the inner level names the reduction. To apply *different* reductions to different columns, pass a dict:

```
df.groupby('key').aggregate({'val1': 'min',
                             'val2': 'max'})
```

```
     val1  val2
key
A       0     5
B       1     7
C       2     9
```

6.1.1 Named aggregation

Modern Pandas offers *named aggregation*, which lets you choose the output column names directly using `pd.NamedAgg` (or the equivalent tuple form). This avoids the MultiIndex columns and is much friendlier downstream:

```
df.groupby('key').agg(
    lo=pd.NamedAgg(column='val1', aggfunc='min'),
    hi=pd.NamedAgg(column='val2', aggfunc='max'),
)
```

```
     lo  hi
key
A     0   5
B     1   7
C     2   9
```

6.2 `filter`: keep or drop *whole* groups

`filter` makes a *group-level* yes/no decision. You supply a function that receives each group's sub-frame and returns a single boolean; groups for which it returns `True` are kept *in full*, the rest are dropped entirely.

```python
def spread_above(group):
    return group['val2'].std() > 2.5

df.groupby('key').filter(spread_above)
```

```
  key  val1  val2
0   A     0     5
2   C     2     3
3   A     3     3
5   C     5     9
```

Intuition

`filter` is the only one of the four families whose *rows* are a subset of the original. It does not transform values and it does not reduce them; it merely *selects which groups survive*, then returns those groups' original rows untouched (note the preserved original index `0, 2, 3, 5`). Use it to answer questions like "show me only the categories with enough variation / enough data / a high enough mean."

6.3 `transform`: same shape in, same shape out

`transform` is the family people forget exists, and it is enormously useful. Unlike `aggregate`, which *collapses* each group to one row, `transform` returns an object the *same shape* as the input, with each value replaced by something computed from its group. The canonical use is *group-wise centering or standardization*:

```python
# Subtract each group's mean from its members (mean-centering)
df.groupby('key')['val2'].transform(lambda x: x - x.mean())
```

```
0   -1.0
1   -3.5
2   -3.0
3    1.0
4    3.5
5    3.0
Name: val2, dtype: float64
```

Key Idea

`aggregate` answers "what is the summary *of* each group?" and shrinks the data. `transform` answers "what does each row look like *relative to* its group?" and preserves the data's shape, so the result lines up row-for-row with the original frame. That alignment is exactly what makes `transform` the right tool for feature engineering: you can drop its output straight back in as a new column, e.g. assigning `df.groupby('key')['val2'].transform(...)` to a fresh column `df['val2_c']`.

6.4 apply: the general escape hatch

When none of the structured families fit, **apply** lets you run an *arbitrary* function on each group's sub-frame. Whatever your function returns, Pandas tries to stitch the pieces back together. Here we normalize `val1` by the group's total of `val2`:

```
def normalize(group):
    group = group.copy()
    group['val1'] = group['val1'] / group['val2'].sum()
    return group

df.groupby('key').apply(normalize)
```

Intuition

apply is maximally flexible and therefore maximally slow and unpredictable: because it must call your Python function once per group and then guess how to recombine arbitrary return values, it cannot use the fused fast paths that `aggregate`, **filter**, and `transform` enjoy. Treat it as the escape hatch: reach for it only when your computation genuinely does not fit the other three. If it does fit, the specialized tool will be clearer *and* faster.

7 Specifying the split key flexibly

So far we have grouped by a single column name. But the "split" phase accepts the key in several forms, and knowing them turns awkward problems into one-liners. All of the following are valid arguments to `groupby`.

Ways to specify the grouping key:

1. **A column name** (or list of names) — group by those columns.
2. **A list/array/Series** of labels aligned to the index — one label per row.
3. **A dict or Series** mapping index values → group names.
4. **A function** applied to each index value to produce its group.
5. **Any combination** of the above in a list.

7.1 A list or array of labels

The key does not have to live in the frame at all. Any array the same length as the frame works — each entry says which group its row belongs to:

```
labels = ['x', 'x', 'y', 'y', 'x', 'y']
df.groupby(labels)['val1'].sum()
```

```
x    5
y   10
Name: val1, dtype: int64
```

7.2 A dict or Series mapping index -> group

If the information you want to group by is a property of the *index*, a mapping is cleanest. Suppose the index labels are codes and you have a lookup from code to category:

```
df2 = df.set_index('key')          # index is now A,B,C,A,B,C
mapping = {'A': 'vowel-ish', 'B': 'consonant',
           'C': 'consonant'}
df2.groupby(mapping)['val1'].sum()
```

```
consonant    12
vowel-ish     3
Name: val1, dtype: int64
```

7.3 A function of the index

Pass any function and Pandas calls it on each index value to compute that row's group. A common idiom is to fold case or bucket the index:

```
# Group by the lowercased first letter of the index label
df2.groupby(str.lower)['val1'].sum()
```

```
a    3
b    5
c    7
Name: val1, dtype: int64
```

7.4 Combining keys for a MultiIndex result

Finally, pass a *list* of any of the above to group hierarchically. The result is indexed by a `MultiIndex`, one level per key:

```
df2.groupby([str.lower, mapping])['val1'].sum()
```

```
a  vowel-ish    3
b  consonant    5
c  consonant    7
Name: val1, dtype: int64
```

Common Pitfall

When you group by a *column* of the frame, that column is consumed and reappears as the result's index. But when you group by an external array, dict, or function, the original columns are untouched — and if you forget to select a numeric subset, Pandas may try to aggregate non-numeric columns too (or, in recent versions, raise rather than silently dropping them). Be explicit: index the column(s) you mean to aggregate, as in `groupby(...)['val1']`.

Section recap

- Simple aggregations (**`sum`**, `mean`, `std`, `describe`, ...) collapse an axis; `axis=0` eats rows (one value per column), `axis=1` eats columns. They skip `NaN` by default.
- **Split–apply–combine** (Wickham) is the organizing idea: split rows into groups by a key, apply a function within each, combine the results into a key-indexed output.
- `groupby(...)` is **lazy** — it returns a `GroupBy` recipe that computes nothing until you aggregate, which is what makes it efficient and chainable.
- A `GroupBy` supports column indexing, iteration over `(key, sub-frame)` pairs, and method dispatch (e.g. per-group `describe()`).
- Four operation families, distinguished by output shape: `aggregate` (one row per group), **`filter`** (a subset of whole groups), `transform` (same shape, group-relative values), **`apply`** (arbitrary, the escape hatch).
- The split key is flexible: a column name, an aligned array/Series, a dict/Series mapping the index, a function of the index, or a list combining these for a `MultiIndex` result.